Motion Retargeting

Homework 2 | C++17 | Skeleton A to Skeleton B

Transfer a captured walk to a new body.

Implement the transformation pipeline yourself:

01 Quaternion math

02 Forward kinematics

03 CCD inverse kinematics

04 Retargeting

Base score: 100 points **Optional bonus: +15 points**

Build and run

No third-party C++ dependencies

- Requirements: CMake 3.16+ and a C++17 compiler.
- Configure and build: cmake -S . -B build ; cmake --build build --config Release
- Linux/macOS: ./build/hw2 data/source.motion data/target.rig build/viewer.html
- Windows: build\Release\hw2.exe data\source.motion data\target.rig build\viewer.html

Run commands from the project folder. A single-config Windows build may use build\hw2.exe.

Data and coordinate conventions

The starter bundles one real CMU capture and a generated target rig

- A: data/source.motion is converted from CMU subject 02, trial 01 (walk). It has 31 joints and 172 frames at about 60 fps.
- B: data/target.rig has the same parent structure, names prefixed B_, and different limb and torso lengths.
- Y is up. Positions and offsets are in meters; source BVH values were multiplied by 0.0254 during preparation.
- Each frame stores the root translation and one local quaternion per joint. Quaternion order is (w, x, y, z).
- The rig is listed in parent-before-child order. A joint offset is expressed in its parent's local frame.

Tasks 1 and 2: math and FK

Implement the TODOs in src/math.cpp and src/fk.cpp

- Math: normalize a quaternion, compute Hamilton multiplication, build an axis-angle quaternion, and rotate a vector.
- Composition convention: a*b applies b first, then a. Identity quaternion = (1, 0, 0, 0).
- FK root: P[0] = rootTranslation + offset[0]; Q[0] = normalized localRotation[0].
- FK child j with parent p: P[j] = P[p] + rotate(Q[p], offset[j]); Q[j] = normalize(Q[p] * localRotation[j]).
- Do not use the rest-pose fallback in the starter as a final FK solution. Test a rotated parent and verify its child moves.

Tasks 3 and 4: IK and retargeting

Implement the TODOs in src/ik.cpp and src/retarget.cpp

- Retarget: map each B_<name> joint to the source joint named <name>; transfer and normalize the local rotation.
- Scale the root translation by 1.12. Use B's offsets for its distinct bone lengths; throw a clear error on unmatched joints.
- IK: solve the left and right feet with CCD. Rotate each leg's parent joints, from lower leg to upper leg, up to 30 iterations.
- At each CCD step, align the current end-effector vector with the goal vector, convert the world correction to the joint's local frame, then recompute FK.
- Clamp dot products to [-1, 1], handle zero-length vectors, and stop early when the foot is within 1 cm. Unreachable goals should remain finite.

Scoring

100 base points + 15 optional bonus points

Quaternion math normalize 3, multiply 4, axis-angle 4, rotate 4	**15**
Forward kinematics root 5, hierarchy positions 15, rotations 5	**25**
CCD inverse kinematics joint updates 10, convergence/robustness 10, both feet 5	**25**
Retargeting name mapping 8, root scaling 5, pose transfer 7, integration 5	**25**
Report method and formulas 4, validation 4, limitations 2	**10**

Bonus +15: stance-aware foot lock (10) and smooth contact transitions (5).

Report and submission

Follow HW1's folder-style submission format

- Write a short PDF report with your method and formulas, implementation decisions, FK and IK checks, source/target frame comparisons, and limitations.
- Compress the entire hw2_<studentID>/ folder as hw2_<studentID>.zip. The folder itself must be the ZIP's top-level entry; do not zip selected files directly.
- Inside that folder, include src/, CMakeLists.txt, data/, and report_<studentID>.pdf.
- Do not include build/ or IDE cache folders. Keep the supplied motion and target rig so the project can run after extraction.

Bonus 1: stance-aware foot lock

+10 points | Implement applyStanceAwareFootLock in src/bonus.cpp

- For each source foot and frame, compute world-space height and horizontal speed from FK positions and frameTime.
- Mark a stance when the foot is near its clip minimum height and moves slowly. State your height and speed thresholds in the report.
- At contact start, save the target foot goal as a fixed world-space anchor. During stance, send this anchor to CCD; during swing, use the regular per-frame goal.
- Treat left and right feet independently. Reject one-frame contact noise so anchors do not flicker.

Check: a planted foot should drift less with the lock enabled than with it disabled.

Bonus 2: smooth transitions

+5 points | Implement applySmoothContactTransitions in src/bonus.cpp

- For each stance interval, ramp a weight from 0 to 1 at contact entry and from 1 to 0 at exit over several frames.
- Use a smoothstep curve, w(t) = 3t^2 - 2t^3, with t clamped to [0, 1]. Blend IK goals as (1-w) * movingGoal + w * plantedAnchor.
- Export three distinct modes: regular goals, hard foot lock, and smoothly blended foot lock. The two bonus checkboxes should select these modes.
- In the report, compare planted-foot drift and entry/exit motion. Explain the thresholds and transition length you chose.

Motion source: CMU Graphics Lab Motion Capture Database, subject 02, trial 01

https://mocap.cs.cmu.edu/